ecd6aeb5b4206a30
cc3aa1f4aba2c5ad.p

Table of Contents

INTRODUCTION 5

FRAMEWORK 6

WEB APPLICATION FRAMEWORK: 6

Features of Django 7

Django Architecture : 8

Install Django in Separate Environment 10

Django Project 12

Create Django Project 12

Django Project Directory Structure 13

Django Project Directory Structure 13

How to run server 14

How to Stop server 14

How to Start/Create Application 14

Creating One Application inside Project:- 14

Creating Multiple Applications inside Project:- 15

How to Install Application in Our Project 15

Application Directory Structure 16

View 17

Function Based View 17

URL Dispatcher 18

URL Pattern inside Project 19

Why URL Pattern inside Application 20

include ( ) 20

Write URL Pattern inside Application ..... **Error! Bookmark not defined.**

Template 23

Create Template Folder and Files 23

Add Templates in settings.py 24

Write Templates Files 24

Rendering Templates Files 26

Django Template Language 30

Variables 30

Filters 31

if Tag 33
if Tag with condition 33
if else Tag 34
if elif Tag with condition 34
for loop Tag 35
Static Files 36
Use Static Files in Template Files 36
Template Inheritance/ Template Extending 37
extends Tag 37
block Tag 38
Creating Base/Parent Template and Child Template 38
Include Tag 40
url Tag 41
Object Relational Mapper (ORM) 42
QuerySet 42
Model 42
Model Class 43
Create Our Own Model Class 43
Migrations 44
Built-in Field Options 45
Writing Code to get DB data in views.py 47
Get Data from views.py in template file 48
QuerySet API 49
Methods that return new QuerySets 49
Operators that return new QuerySets 50
Field Lookups 53
Limiting QuerySets 56
Admin Application 58
Create Super User 58
How to Register Model 58
Example:- 59
ModelAdmin 59
Register Above Created Class 60

Register Model by Decorator 60
Aggregation 61
Django Form 62
Bound and Unbound Forms 63
Create Django Form using Form Class 63
Display Form to User 64
Form Rendering Options 65
Ordering Form Field 66
Form Field 68
Widget 68
GET and POST 70
What is Cross Site Request Forgery (CSRF/ XSRF) 71
Get Django Form Data 72
How to send POST Request 72
Django Form and Field Validation 72
Get Django Form Data in Terminal 73
Form Field 74
Cleaning and Validating Specific Field 76
Validation of Complete Django Form at once 77
Using Built-in Validators 78
Create Custom Form Validators 78
Model Form 79
Authentication and Authorization 81
User Authentication System 82
User object Fields 83
authenticate ( ) 84
login ( ) 84
logout ( ) 84
Type of Views 85
Class Based View 85
Advantages:- 85
Class Based View:- 85
Model Inheritance 86

Abstract Base Classes 86
Multi-table Inheritance 88
Proxy Model 88

INTRODUCTION

Django is a free and open source web application framework, written in Python. It is maintained by the Django Software Foundation (DSF)

- ☐ Encourages rapid development and clean, pragmatic design.
- ☐ Two Web programmers at the Lawrence Journal-World newspaper, **Adrian Holovaty** and **Simon Willison.**
- ☐ Named after famous Guitarist Django Reinhardt
- ☐ Created in 2003, open sourced in 2005

1.0 Version released in 2008, 2.1 Version released in 2018, 3.0 Version released in 2019,

4.0 Version released in 2021 ,5.0 version released in 2023, and the latest version is 6.0 released in 2025

FRAMEWORK

It is a platform for developing software applications. It include code libraries, a compiler, and other programs.

WEB APPLICATION FRAMEWORK:

- A Web Framework (WF) or Web Application Framework (WAF) which helps to build Web Applications.
- Web frameworks provide tools and libraries to simplify common web development operations. This can include web services, APIs, and other resources.
- Web frameworks help with a variety of tasks, from templating and database access to session management and code reuse.
- More than 80% of all web app frameworks rely on the Model View Controller architecture.

• PYTHON ->

Django, TurboGears, Pylons, Zope, Quixote, webzpy, Google App Engine, CherryPY, Pyramid, Flask

• Ruby -> RubyOnRails(RoR),

• PERL -> Catalyst, Mason,

• JAVASCRIPT->JavaScriptMVC+

• PHP -> CakePHP, CodeIgniter, PRADO, ThinkPHP, QeePHP

• ASP -> ASP.NET, ASP MVC

• JAVA -> Spring, Apache, Struts, Spring, Tapestry, GWT

Features of Django

- **Ridiculously fast.** :

Django was designed to help developers take applications from concept to completion as quickly as possible.

- **Fully loaded.** :

Django includes dozens of extras you can use to handle common web development tasks. Django takes care of user authentication, content administration, site maps, RSS feeds, and many more tasks — right out of the box.

- **Reassuringly secure.** :

Django takes security seriously and helps developers avoid many common security mistakes, such as SQL injection, cross-site scripting, cross-site request forgery and clickjacking. Its user authentication system provides a secure way to manage user accounts and passwords.

- **Exceedingly scalable**. :

Some of the busiest sites on the planet use Django's ability to quickly and flexibly scale to meet the heaviest traffic demands.

- **Incredibly versatile.** :

Companies, organizations and governments have used Django to build all sorts of things — from content management systems to social networks to scientific computing platforms.

Django Architecture :

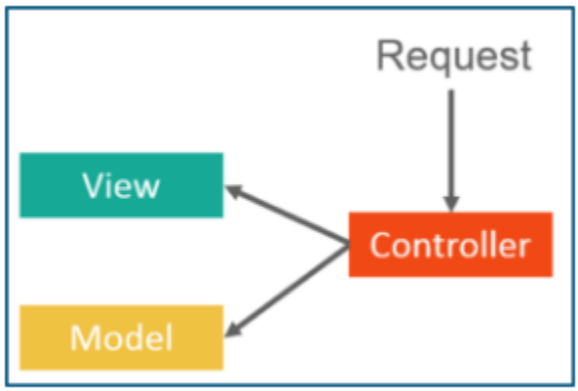
Django follows a MVC- MVT architecture.

MVC stands for Model View Controller. It is used for developing the web application, where we break the code into various segments. Here we have 3 segments, model view and a controller.

Model – Model is used for storing and maintaining your data. It is the backend where your database is defined.

Views – In Django templates, views are in html. View is all about the presentation and it is not at all aware of the backend. Whatever the user is seeing, it is referred to a view.

Controller – Controller is a business logic which will interact with the model and the view.



Now that we have understood MVC, lets learn about Django MVT pattern.
MVT stands for Model View Template.
In the case of MVT, Django itself takes care of the controller part, it's inbuilt.

Model :

The Model responsible to handle database. It is a data access layer which handles the data

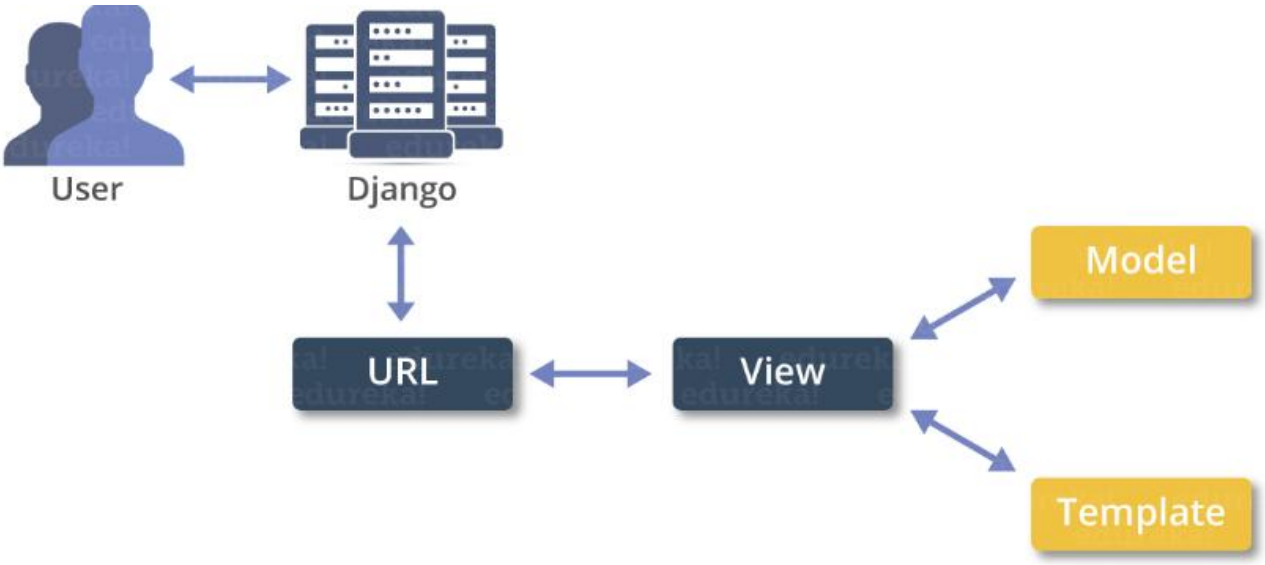
View :

The user can send request by interacting with template, the view handles these requests and sends to Model then get appropriate response from the Model, sends response to template.
It may also have required logics.
It works as a mediator between Template and Model

Template:

A template is a text file. It can generate any text-based format (HTML, XML, CSV, etc.).
A template contains variables, which get replaced with values when the template is evaluated, and tags, which control the logic of the template.
Template is used by view function to represent the data to user.

User sends request to view then view contact template after that view get information from the template and then view gives response to the users.



Django Requirements

- Python 3.0 or Higher

  python --version

- PIP

  pip --version

- Text/Code Editor/IDE – Notepad++, VS Code, ATOM, Brackets, PyCharm
- Web Browser – Google Chrome, Mozilla Firefox, Edge

Install Django in Separate Environment

Create Virtual Environment (VE) :

py -m venv virtual_environment_name

or

python -m venv virtual_environment_name

This is used to create virtual environment

Example

py -m venv env

to activate virtual environment

to activate we need to go to scripts folder which is present inside virtual environment folder

step1 : cd virtual_environment_name/scripts

step2 : activate

after activating come back to project folder

command

cd ../..

Install Django in Created VE

Command

pip install django

First activate environment then run the command to install django within active environment. You can also specify version *pip install* ***django==5.0***

Check Django is installed or Not

django-admin

it will display all sub commands

Check Django Version

django-admin --version

Django Project

A Django Project may contain multiple Project Application, which means a group of Application and files is called as Django Project.

An Application is a Part of Django Project.

School

- Registration App
- Fees App
- Exam App
- Attendance App
- Result App

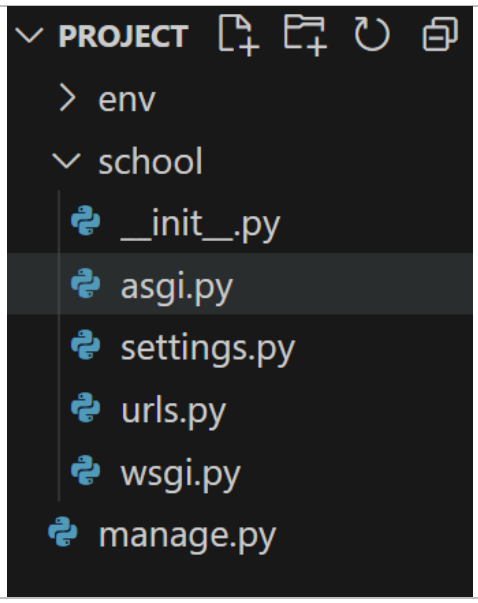
Create Django Project

Syntax: -

django-admin startproject projectname .

Example:-

django-admin startproject school .

project ***school*** *__init__.py* *asgi.py* *settings.py* *urls.py* *wsgi.py* *manage.py*	

Django Project Directory Structure

Project → ***Outer Project Folder/Root Directory***

School → ***Inner Project Folder***

__init__.py

asgi.py

settings.py

urls.py

wsgi.py

manage.py

__init__.py – The folder which contains __init__.py file is considered as python package.

wsgi.py – WSGI (Web Server Gateway Interface) is a specification that describes how a web server communicates with web applications, and how web applications can be chained together to process one request. WSGI provided a standard for synchronous Python apps.

asgi.py – ASGI (Asynchronous Server Gateway Interface) is a spiritual successor to WSGI, intended to provide a standard interface between async-capable Python web servers, frameworks, and applications. ASGI provides standard for both asynchronous and synchronous apps.

settings.py – This file contains all the information or data about project settings.

E.g.:- Database Config information, Template, Installed Application, Validators etc.

urls.py – This file contains information of url attached with application.

manage.py – manage.py is automatically created in each Django project. It is Django's command-line utility also sets the DJANGO_SETTINGS_MODULE environment variable so that it points to your project's settings.py file. Generally, when working on a single Django project, it's easier to use manage.py than django-admin.

How to run server

Django provides built-in server which we can use to run our project.

runserver – This command is used to run built-in server of Django.

Steps:-

- Go to Project Folder
- Then run command ***python manage.py runserver***
- Server Started
- Visit *http://127.0.0.1:8000* or *http://localhost:8000*
- You can specify Port number ***python manage.py runserver 5555***
- Visit *http://127.0.0.1:5555* or *http://localhost:5555*

How to Stop server

ctrl + c is used to stop Server

Note – Sometime when you make changes in your project you may need to restart the server.

How to Start/Create Application

A Django project contains one or more applications which means we create application inside Project Folder.

Syntax:-

django-admin **startapp appname**

or

python manage.py **startapp appname**

Creating One Application inside Project:-

- Go to Project Folder/root folder
- Run Command ***django-admin startapp course***

Creating Multiple Applications inside Project:-

- Go to Project Folder/root folder
- ***django-admin startapp course***
- ***django-admin startapp fees***
- ***django-admin startapp result***

How to Install Application in Our Project

As we know a Django project can contain multiple application so just creating application inside a project is not enough we also have to install application in our project.

We install application in our project using settings.py file.

settings.py file is available at Project Level which means we can install all the application of project.

This is compulsory otherwise Application won't be recognized by Django.

Open **settings.py** file

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'application_name1',
    'application_name2',
]
```

Example

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'course',
    'fees',
    'result'
]
```

Application Directory Structure

App_folder

- **migrations**
  - __init__.py
- __init__.py
- admin.py
- apps.py
- models.py
- tests.py
- views.py

migrations – This folder contains __init__.py file which means it's a python package. It also contains all files which are created after running makemigration command .

__init__.py – The folder which contains __init__.py file is considered as Python Package.

admin.py – This file is used to register sql tables so we could perform CRUD operation from Admin Application. Admin Application is provided by Django to perform CRUD operation.

apps.py – This file is used to config app.

models.py – This file is used to create our own model class later these classes will be converted into database table by Django for our application.

tests.py – This is files is used to create tests.

views.py – This file is used to create view. We write all the business logic related code in this file.

View

- Function Based View
- Class Based View

Function Based View

A function based view, is a Python function that takes a Web request and returns a Web response.

This response can be the HTML contents of a Web page, or a redirect, or a 404 error, or an XML document, or an image or anything.

Each view function takes an HttpRequest object as its first parameter.

The view returns an HttpResponse object that contains the generated response. Each view function is responsible for returning an HttpResponse object.

We will call these functions as *view function* or *view function of application or view*.

Syntax

```
def function_name (request):
    return HttpResponse('html/variable/text')
```

We use **views.py** file of the application to write functions which may contain business logic of application, later it required to define url name for this function in the **urls.py** file of the project.

Where **HttpResponse** is class which is in **django.http** module so we have to import it before using **HttpResponse**.

views.py

```
from django.http import HttpResponse

def learn_django(request):
    return HttpResponse('Hello Django')


def learn_python(request):
    return HttpResponse('<h1>Hello Python</h1>')
```

URL Dispatcher

To design URLs for app, you create a Python module informally named urls.py This module is pure Python code and is a mapping between URL path expressions to view functions.

This mapping can be as short or as long as needed.

It can reference other mappings.

It's pure Python code so it can be constructed dynamically.

path ( )

path(route, view, name=None) - It returns an element for inclusion in urlpatterns.

Where,

- The route argument should be a string that contains a URL pattern. The string may contain angle brackets e.g. <username> to capture part of the URL and send it as a keyword argument to the view. The angle brackets may include a converter specification like the int part of <int:id> which limits the characters matched and may also change the type of the variable passed to the view. For example, <int:id> matches a string of decimal digits and converts the value to an int.
- The view argument is a view function or the result of as_view() for class-based views. It can also be an django.urls.**include()**.

urls.py

```
urlpatterns = [
        path(route, view, name=None)
        ]
```

urls.py

```
urlpatterns = [
        path('learndj/', views.learn_Django, name='learn_django'),
        ]
```

URL Pattern inside Project

urls.py file is used to define url pattern attached with application or view of application or view function of application.

urls.py file is located inside *inner project folder* not inside *application folder* which means we define url at project level for applications. Defined url name will be used by application user to get response from the application or view function of application.

Steps:-

- Open urls.py
- Import Module (Python file) of the application
- Write URL Name and Map it with function

We can define multiple url for one view function. Which means we can access same view function with multiple urls.

urls.py

from course import views

urlpatterns = [

path('learndj/', views.learn_django),

path('altlearndj/', views.learn_django),

]

http://127.0.0.1:8000/learndj

http://127.0.0.1:8000/altlearndj

views.py of course

from django.http import HttpResponse

def learn_django(request):

return HttpResponse('Hello Django')

views.py of fees

from django.http import HttpResponse

def fees_django(request):

return HttpResponse('300')

urls.py

```
from course import views as cv

from fees import views as fv

urlpatterns = [

        path('learndj/', cv.learn_django),

        path('feesdj/', fv.fees_django),

]
```

Why URL Pattern inside Application

So far we have learnt to define url pattern at project level for our all application.

This approach increases the dependency of applications in project which means if we want to use a particular application for our another project we may face issues.

Our each application should be independent or less depend on project so we could use our applications in different projects easily without worrying about urls.py available in Project Folder.

Following are the benefits of defining url pattern inside Application

- Reduces the dependency of Application
- Enhance Application performance.
- Reusability of application becomes easy

include ( )

A function that takes a full Python import path to another URLconf module that should be "included" in this place.

Optionally, the application namespace and instance namespace where the entries will be included into can also be specified.

include() also accepts as an argument either an iterable that returns URL patterns or a 2-tuple containing such iterable plus the names of the application namespaces.

urlpatterns can "include" other URLconf modules.

Syntax:-

include(‘app_name.urls’)

Where,

module – URLconf module (or module name)

namespace (str) – Instance namespace for the URL entries being included

pattern_list – Iterable of path()

app_namespace (str) – Application namespace for the URL entries being included

Example:-

```
from django.urls import include, path
urlpatterns = [
        path(‘cor/', include(‘course.urls')),
]
```

- Create an urls.py file inside each application (in case multiple application).
- Write all url pattern related to application, in urls.py file available inside application.
- Include Application’s urls.py file inside Project’s urls.py file.

views.py in Application Folder

```
from django.http import HttpResponse
def learn_django(request):
        return HttpResponse(‘Hello Django’)
def learn_python(request):
        return HttpResponse(‘<h1>Hello Python</h1>’)
```

urls.py in Application Folder

```
from course import views
urlpatterns = [
        path('learndj/', views.learn_django),
        path('learnpy/', views.learn_python),
]
```

urls.py in Project Folder

```
from django.urls import path, include
urlpatterns = [
        path('cor/', include('course.urls')),
]
```

http://127.0.0.1:8000/cor/learndj

http://127.0.0.1:8000/cor/learnpy

Template

A template is a text file. It can generate any text-based format (HTML, XML, CSV, etc.).

A template contains variables, which get replaced with values when the template is evaluated, and tags, which control the logic of the template.

Template is used by view function to represent the data to user.

User sends request to view then view contact template after that view get information from the template and then view gives response to the users.

Create Template Folder and Files

We create **templates** folder inside Project Folder. **templates** folder will contain all html files.

project

templates  **This is templates Folder**

courseone.html

coursetwo.html **Template files**

feesone.html

feestwo.html

school

__init__.py

settings.py

urls.py

wsgi.py

manage.py

course

fees

Add Templates in settings.py

TEMPLATES_DIR = os.path.join(BASE_DIR, 'templates')

Or

TEMPLATES_DIR = BASE_DIR / 'templates'

```
TEMPLATES = [
    {
        'DIRS': [TEMPLATES_DIR],
    }
]
```

Write Templates Files

When we create Template file for application we separate business logic and presentation from the application *views.py* file.

Now we will write business logic in *views.py* file and presentation code in template file.

courseone.html

```
<html>
    <head>
        <style> ……. </style>
    </head>
    <body>
        <h1>Hello Django</h1>
    </body>
    <script>…….</script>
</html>
```

coursetwo.html

```
<html>
    <body>
        <h1>Hello Python</h1>
    </body>
</html>
```

feesone.html

```
<html>
    <body>
        <h1>300</h1>
    </body>
</html>
```

feestwo.html

```
<html>
    <body>
        <h1>200</h1>
    </body>
</html>
```

Rendering Templates Files

By Creating Template file for application we separate business logic and presentation from the application *views.py* file. Now we will write business logic in views.py file and presentation code in html file.

Still *views.py* will be responsible to process the template files for this we will use *render()* function in *views.py* file.

render()

It combines a given template with a given context dictionary and returns an HttpResponse object with that rendered text.

Syntax:-

```
render(request, template_name, context=dict_name)
```

Where,

request – The request object used to generate this response.*

template_name – The full name of a template to use or sequence of template names. If a sequence is given, the first template that exists will be used. *

context – A dictionary of values to add to the template context. By default, this is an empty dictionary. If a value in the dictionary is callable, the view will call it just before rendering the template.

views.py

```
from django.shortcuts import render

def function_name(request):

        Dynamic Data, if else, any python code logic

        return render(request, template_name, context=dict_name)
```

courseone.html

```
<html>
    <head>
       <style> ……. </style>
    </head>
    <body>
        <h1>Hello Django</h1>
    </body>
    <script>…….</script>
</html>
```

views.py

```
from django.shortcuts import render
def learn_django(request):
          return render(request, 'courseone.html')
```

Dynamic Template Files using DTL

courseone.html

```
<html>
    <head>
       <style> ……. </style>
    </head>
```

```
    <body>
        <h1>Welcome to GeekyShows</h1>
        <h1>Hello {{cname}}</h1>    {{cname}}    variable
</body>
    <script>…….</script>
</html>
```

views.py

```
from django.shortcuts import render
def function_name(request):
    Dynamic Data, if else, any python code logic
    return render(request, template_name, context=dict_name)
```

```
from django.shortcuts import render
def learn_django(request):
        coursename = {'cname': 'Django'}
        return render(request, 'course/courseone.html', context=coursename)
                                    or
        return render(request, 'course/courseone.html', coursename)
                                    or
        return render(request, 'course/courseone.html',{'cname': 'Django'})
```

example

views.py

```
from django.shortcuts import render
def learn_django(request):
        cname = 'Django'
```

```
        duration = '4 Months'
        seats = 10
        django_details = {'nm':cname, 'du':duration, 'st':seats}
        return render(request, 'course/courseone.html', django_details)
```

templates/course

***courseone*.html**

```
<html>
  <body>
        <h2> Course Name:{{nm}} Duration:{{du}} and Total Seats: {{st}}</h2>
   </body>
</html>
```

Django Template Language

Django's template language is designed to strike a balance between power and ease. It's designed to feel comfortable to those used to working with HTML.

***courseone*.html**

```
<html>
  <body>
        <h2> Course Name:{{nm}} Duration:{{du}} and Total Seats: {{st}}</h2>
   </body>
</html>
```

Jinja2 - Jinja is a modern and designer-friendly templating language for Python, modelled after Django's templates. It is fast, widely used and secure with the optional sandboxed template execution environment.

python pip install jinja2

'BACKEND': 'django.template.backends.jinja2.Jinja2

Variables

Variables look like this: **{{ variable }}** When the template engine encounters a variable, it evaluates that variable and replaces it with the result.

Rules:-

Variable names consist of any combination of alphanumeric characters and the underscore.

Variable name should not start with underscore.

Variable name can not have spaces or punctuation characters.

Syntax:- {{variable}}

Example:- {{nm}}, {{name1}}, {{first_name}}

Filters

When we need to modify variable before displaying we can use filters. Pipe '**|**' is used to apply filter.

Syntax:- {{variable **|** filter}}

Example:- {{name**|**upper}}

Some of filters take arguments.

Syntax:- {{variable **|** filter **:** argument}}

Example:- {{article**|**truncateword**:**40}}

Filter can be chained.

Syntax:- {{variable **|** filter **|** filter}}

Example:- {{article|upper}}

Example:- {{article**|**upper**|**truncateword**:**40}}

capfirst – It capitalizes the first character of the value. If the first character is not a letter, this filter has no effect.

Example:- {{value|capfirst}}

default - If value evaluates to False, uses the given default. Otherwise, uses the value.

Example:- {{ value|default:"nothing" }}

If value is "" (the empty string), the output will be nothing.

length - It returns the length of the value. This works for both strings and lists. The filter returns 0 for an undefined variable.

Example:- {{ value|length }}

lower - It converts a string into all lowercase.

Example:- {{ value|lower }}

upper - It converts a string into all uppercase.

Example:- {{ value|upper }}

slice - It returns a slice of the list. Uses the same syntax as Python's list slicing.

Example:- {{ some_list|slice:":2" }}

truncatechars - It truncates a string if it is longer than the specified number of characters. Truncated strings will end with a translatable ellipsis character ("…").

Argument: Number of characters to truncate to

Example:- {{ value|truncatechars:7 }}

truncatewords - It truncates a string after a certain number of words. Newlines within the string will be removed.

Argument: Number of words to truncate after

Example:- {{ value|truncatewords:2 }}

date – It formats a date according to the given format.

Example:- {{value|date:"D d M Y"}}

time – It formats a time according to the given format.

Example:- {{value|time:"H:i"}}

if Tag

{% if %} tag - The {% if %} tag evaluates a variable, and if that variable is “true” (i.e. exists, is not empty, and is not a false boolean value).

Syntax:-

```
{% if variable %}

        ……..

{% endif %}
```

Example:-

```
{% if nm %}

        </h1>Hello {{nm}}</h1>

{% endif %}

{% if nm and st %}

        </h1> For Course{{nm}} {{st}} Seat Available</h1>

{% endif %}
```

if Tag with condition

Syntax:-

```
{% if condition %}

        ……..

{% endif %}
```

Example:-

```
{% if nm == ‘Django’  %}

        </h1>Hello {{nm}}</h1>

{% endif %}

{% if nm == ‘Django’ and st==5 %}

        </h1>{{nm}} Seat Available</h1>

{% endif %}
```

if tags may also use the operators **==, !=, <, >, <=, >=, in, not in, is,** and **is not**

if else Tag

Syntax:-

```
{% if variable %}
        ……..
{% else %}
        ……..
{% endif %}
```

Example:-

```
{% if nm %}
        </h1>Hello {{nm}}</h1>
{% else}
        <h1>No Course Available</h1>
{% endif %}
```

if elif Tag with condition

Syntax:-

```
{% if condition %}
        ……..
{% elif condition %}
        ……..
{% else %}
        ……..
{% endif %}
```

Example:-

```
{% if nm==‘Django’ %}
        </h1>Hello {{nm}}</h1>
{% elif st==5 %}
```

```
        </h1>Seats {{st}}</h1>
{% else}
        <h1>No Course Available</h1>
{% endif %}
```

for loop Tag

Syntax:-

```
{% for variable in variables %}
   {{ variable }}
{% endfor %}
```

Example:-

```
<ul>
{% for stu in student %}
   <li>{{ stu }}</li>
{% endfor %}
</ul>
```

Static Files

CSS files, Javascript Files, image files, video files etc are considered as static files in Django.

Django provides django.contrib.staticfiles to help you manage them.

django.contrib.staticfiles collects static files from each of your applications (and any other places you specify) into a single location that can easily be served in production.

How to Create Static Folder and Files

We create **static** folder inside Root Project Folder then inside **static** folder we create required folders which will contain all static files respectively like css folder will contain all css files, js folder will contain all js files and image folder will contain all img and so on.

Use Static Files in Template Files

- First Load Static Files
- Reference Static Files

templates/course

courseone.html

```
<!DOCTYPE html>
{% load static %}                    // Loading Static Files
<html>
    <link href='{% static "css/style_name.css" %}'>
    <body>
        <h1>Fees {{fe}} </h1>
        <img src='{% static "img/img_name.jpg" %}'>
        <script src = "{% static 'js/filename.js' %}" > </script>
    </body>
</html>
```

Template Inheritance/ Template Extending

Template inheritance allows you to build a base “skeleton” template that contains all the common elements of your site and defines blocks that child templates can override.

The *extends* tag is used to inherit template.

extends tag tells the template engine that this template “extends” another template.

When the template system evaluates this template, first it locates the parent let’s assume, “base.html”.

At that point, the template engine will notice the block tags in base.html and replace those blocks with the contents of the child template.

You can use as many levels of inheritance as needed.

extends Tag

{% extends %} – The *extends* tag is used to inherit template. It tells the template engine that this template “extends” another template. It has no end tag.

Syntax:-

{% extends ‘parent_template_name’ %}

{% extends variable %}

Example:-

{% extends "./base1.html" %}

{% extends "../base2.html" %}

block Tag

{% block %} – The *block* tag is used to for overriding specific parts of a template.

Syntax:-

{% block blockname %}….{% endblock %}

{% block blockname %}….{% endblock blockname %}

Example:-

{% block title %} ….. {% endblock %}

{% block content %}….. {% endblock content %}

Rules

- If We use {% extends %} in a template, it must be the first template tag in that template. Template inheritance won't work, otherwise.
- More {% block %} tags in our base templates are better.
- Child templates don't have to define all parent blocks, so we can fill in reasonable defaults in a number of blocks, then only define the ones we need later.
- We Can't define multiple block tags with the same name in the same template.
- If We need to get the content of the block from the parent template, the {{ block.super }} variable will do the trick.

Creating Base/Parent Template and Child Template

We write common codes in base template and create blocks for code which may vary page to page. Later this template will be inherited by child templates and child template will override created blocks.

home.html

```
<html>
<head>
<title>Home</title>
</head>
<body>
<h1>Hello I am Home Page</h1>
</body>
</html>
```

about.html

```
<html>
<head>
```

```
<title>About</title>
</head>
<body>
<h1>Hello I am About Page</h1>
</body>
</html>
```

base.html

```
<html>
<head>
<title>{% block title %} {% endbock %}</title>
</head>
<body>
{% block content %} {% endblock content %}
</body>
</html>
```

home.html

```
{%  extends 'base.html' %}
{% block title %}
  Home
{% endblock %}
{% block content %}
  <h1>Hello I am Home Page </h1>
{% endblock content %}
```

about.html

```
{%  extends 'base.html' %}
{% block title %}
```

```
  About
{% endblock %}
{% block content %}
  <h1>Hello I am About Page </h1>
{% endblock content %}
```

Include Tag

{% include %} Tag – It loads a template and renders it with the current context. This is a way of "including" other templates within a template. Each include is a completely independent rendering process.

The template name can either be a variable or a hard-coded (quoted) string, in either single or double quotes.

Syntax:-

```
{% include temp_var_name %}
{% include "template_name.html" %}
{% include "folder/template_name.html" %}
```

Example:-

```
{% include topvar %}
{% include "topcourse.html" %}
{% include "fees/extrafees.html" %}
```

url Tag

{% url %} - It returns an absolute path reference (a URL without the domain name) matching a given view and optional parameters.

Syntax:-

{% url 'urlname' %}

{% url 'urlname' arg1=value1 arg2=value2 %}

{% url 'urlname' value1 value2 %}

Example:

```
def about(request):
        return render(request, 'core/about.html')
urlpatterns  =  [
                  path('about/', views.about, name='aboutus’)
                 ]
```

```
<a href="{% url 'aboutus' %}">About</a>
```

Object Relational Mapper (ORM)

Object-Relational Mapper (ORM), which enables application to interact with database such as SQLite, MySQL, PostgreSQL, Oracle.

ORMs automatically create a database schema from defined classes or models. It generate SQL from Python code for a particular database which means developer do not need to write SQL Code.

ORM maps objects attributes to respective table fields.

It is easier to change the database if we use ORMs hence project becomes more portable.

Django's ORM is just a way to create SQL to query and manipulate your database and get results in a pythonic fashion.

ORMs use connectors to connect databases with a web application.

QuerySet

A QuerySet can be defined as a list containing all those objects we have created using the Django model.

QuerySets allow you to read the data from the database, filter it and order it

Model

A model is the single, definitive source of information about your data.

It contains the essential fields and behaviors of the data you're storing.

Generally, each model maps to a single database table

Model Class

Model class is a class which will represent a table in database.

Each model is a Python class that subclasses django.db.models.Model

Each attribute of the model represents a database field.

With all of this, Django gives you an automatically-generated database-access API Django provides built-in database by default that is sqlite database.

We can use other database like MySQL, Oracle SQL etc.0

Create Our Own Model Class

models.py file which is inside application folder, is required to create our own model class.

Our own model class will inherit Python's Model Class.

Syntax:-

```
class ClassName(models.Model):
        field_name=models.FieldType(arg, options)
```

Example:-

models.py

```
class Student(models.Model):
        stuid=models.IntegerField()
        stuname=models.CharField(max_length=70)
        stuemail=models.EmailField(max_length=70)
        stupass=models.CharField(max_length=70)
```

- This class will create a table with columns and their data types
- Table Name will be ApplicationName_ClassName, in this case it will be enroll_student
- Field name will become table's Column Name, in this case it will be stuid, stuname, stuemail, stupass with their data type.
- As we have not mentioned primary key in any of these columns so it will automatically create a new column named 'id' Data Type Integer with primary key and auto increment.

Rules

- Field Name instantiated as a class attribute and represents a particular table's Column name.
- Field Type is also known as Data Type.
- A field name cannot be a Python reserved word, because that would result in a Python syntax error.
- A field name cannot contain more than one underscore in a row, due to the way Django's query lookup syntax works.
- A field name cannot end with an underscore.

Migrations

Migrations are Django's way of propagating changes you make to your models (adding a field, deleting a model, etc.) into your database schema.

- **makemigrations** – This is responsible for creating new migrations based on the changes you have made to your models.
- **migrate** – This is responsible for applying and unapplying migrations.

makemigrations and migrate

- **makemigrations** is used convert model class into sql statements. This will also create a file which will contain sql statements. This file is located in Application's migrations folder.

  Syntax:- python manage.py makemigrations

- **migrate** is used to execute sql statements generated by makemigrations. This command will execute All Application's (including built-in applications) SQL Statements if available. After execution of sql statements table will be created.

  Syntax:- python manage.py migrate

Note – If you make any change in your own model class you are required to run makemigrations and migrate command only then you will get those changes in your application.

Built-in Field Options

null It can contain either True or False. If True, Django will store empty values as NULL in the database. Default is False.

Avoid using null on string-based fields such as CharField and TextField. If a string-based field has null=True, that means it has two possible values for "no data": NULL, and the empty string.

Example:- null=True/False

blank - It can contain either True or False. If True, the field is allowed to be blank. This is different than null. null is purely database-related, whereas blank is validation-related. If a field has blank-True, form validation will allow entry of an empty value. If a field has blank=False, the field will be required. Default is False.

default - The default value for the field. This can be a value or a callable object. If callable it will be called every time a new object is created.

Example:- default='Not Available'

verbose_name - A human-readable name for the field. If the verbose name isn't given, Django will automatically create it using the field's attribute name, converting underscores to spaces.

Example:- verbose_name='Student Name'

primary_key - If True, this field is the primary key for the model.

If you don't specify primary_key=True for any field in your model, Django will automatically add an AutoField to hold the primary key, so you don't need to set primary_key=True on any of your fields unless you want to override the default primary-key behavior.

The primary key field is read-only. If you change the value of the primary key on an existing object and then save it, a new object will be created alongside the old one. primary_key=True implies null=False and unique=True. Only one primary key is allowed on an object.

Example:- primary_key=True

datatypes

-> **IntegerField** - An integer. Values from -2147483648 to 2147483647 are safe in all databases supported by Django. It uses MinValue Validator and Max Value Validator to validate the input based on the values that the default database supports. The default form widget for this field is a NumberInput when localize is False or TextInput otherwise.

Example:- roll = models. IntegerField()

-> **BigIntegerField** - A 64-bit integer, much like an IntegerField except that it is guaranteed to fit numbers from -9223372036854775808 to 9223372036854775807. The default form widget for this field is a TextInput

Example:- mobile = models. BigIntegerField()

-> **CharField** - A string field, for small- to large-sized strings. For large amounts of text, use TextField. The default form widget for this field is a TextInput. CharField has one extra required argument: max_length

The maximum length (in characters) of the field.

Example:- first_name = models.CharField(max_length=30)

-> **TextField** - A large text field. The default form widget for this field is a Textarea. If you specify a max_length attribute, it will be reflected in the Textarea widget of the auto- generated form field. However it is not enforced at the model or database level. Use a CharField for that.

Example :- description = models.TextField()

EmailField - A CharField that checks that the value is a valid email address using EmailValidator.

Example:- email = models.EmailField()

Show Table Data to User

- Writing Code to get data from database in views.py then pass it to template files using render function
- Get Data which is passed by render function of views.py file in template file

all ( )

all ( ) – It returns a copy of current QuerySet or QuerySet Subclass.

Syntax:-

ModelClassName.objects.all()

Writing Code to get DB data in views.py

First import your own model class from models.py

Example:-

views.py

```
from enroll.models import Student

def studentinfo(request):
        stud = Student.objects.all()
        return render(request, 'enroll/studetails.html', {'stu':stud})
```

Get Data from views.py in template file

templates/enroll / studetails.html

```
<!DOCTYPE html>
<html>
	<body>
		<h1>Student Page</h1>
		{% if stu  %}
			 <h1>Show Data</h1>
			{%  for st in stu %}
				<h5>{{st.stuid}}</h5>
				<h5>{{st.stuname}}</h5>
			{% endfor %}
		{% else %}
			<h1>No Data</h1>
		{% endif %}
	</body>
</html>
```

QuerySet API

A QuerySet can be defined as a list containing all those objects we have created using the Django model.

QuerySets allow you to read the data from the database, filter it and order it.

query property – This property is used to get sql query of query set.

Syntax:- queryset.query

Methods that return new QuerySets

Retrieving all objects

all ( ) - This method is used to retrieve all objects. This returns a copy of current QuerySet.

Example:- Student.objects.all()

Retrieving specific objects

- filter (**kwargs) - It returns a new QuerySet containing objects that match the given *lookup parameters*. filter() will always give you a QuerySet, even if only a single object matches the query.

Example:- Student.objects.filter(marks=70)

- exclude(**kwargs) - It returns a new QuerySet containing objects that do not match the given *lookup parameters*.

Example:- Student.objects.exclude(marks=70)

order_by(*fields) – It orders the fields.

- 'field' – Asc order
- '-field' – Desc Order

values(*fields, **expressions) - It returns a QuerySet that returns dictionaries, rather than model instances, when used as an iterable. Each of those dictionaries represents an object, with the keys corresponding to the attribute names of model objects.

Operators that return new QuerySets

Q Objects

Q object is an object used to encapsulate a collection of keyword arguments. These keyword arguments are specified as in “Field lookups” .

If you need to execute more complex queries, you can use Q objects.

Q objects can be combined using the & and | operators. When an operator is used on two Q objects, it yields a new Q object.

from django.db.models import Q

& (AND) Operator

Example:- Student.objects.filter(Q(id=6) & Q(roll=106))

| (OR) Operator

Example:- Student.objects.filter(Q(id=6) | Q(roll=108))

~ Negation Operator

Example:- Student.objects.filter(~Q(id=6))

AND (&) - Combines two QuerySets using the SQL AND operator.

Example:-

student_data = Student.objects.filter(id=6) & Student.objects.filter(roll=106)

student_data = Student.objects.filter(id=6, roll=106)

student_data = Student.objects.filter(Q(id=6) & Q(roll=106))

OR (|) - Combines two QuerySets using the SQL OR operator.

Example:-

Student.objects.filter(id=11) | Student.objects.filter(roll=106)

Student.objects.filter(Q(id=11) | Q(roll=106))

Methods that do not return new QuerySets

Retrieving a single object

get ( ) - It returns one single object. If There is no result match it will raise DoesNotExist exception. If more than one item matches the get() query. It will raise MultipleObjectsReturned.

Example:- Student.objects.get(pk=1)

first() - It returns the first object matched by the queryset, or None if there is no matching object. If the QuerySet has no ordering defined, then the queryset is automatically ordered by the primary key.

Example:- student_data = Student.objects.first()

student_data = Student.objects.order_by('name').first()

last() - It returns the last object matched by the queryset, or None if there is no matching object. If the QuerySet has no ordering defined, then the queryset is automatically ordered by the primary key.

exists() - It returns True if the QuerySet contains any results, and False if not. This tries to perform the query in the simplest and fastest way possible, but it does execute nearly the same query as a normal QuerySet query.

Example:-

student_data = Student.objects.all()

print(student_data.exists())

create(kwargs)** - A convenience method for creating an object and saving it all in one step.

Example:-

s = Student(name='Sameer', roll=112, city='Bokaro', marks=60, pass_date='2020-5-4')

s.save(force_insert=True)

s = Student.objects.create(name='Sameer', roll=112, city='Bokaro', marks=60, pass_date='2020-5-4')

update(kwargs)** - Performs an SQL update query for the specified fields, and returns the number of rows matched (which may not be equal to the number of rows updated if some rows already have the new value).

Example:-

student_data = Student.objects.filter(id=12).update(name='Kabir', marks=80)

Update student's city Pass who has marks 60

student_data = Student.objects.filter(marks=60).update(city='Pass')

delete() - The delete method, conveniently, is named delete(). This method immediately deletes the object and returns the number of objects deleted and a dictionary with the number of deletions per object type.

Example:-

Delete One Record

student_data = Student.objects.get(pk=22)

deleted = student_data.delete()

Delete in Bulk

You can also delete objects in bulk. Every QuerySet has a delete() method, which deletes all members of that QuerySet.

Example:- student_data = Student.objects.filter(marks=50).delete()

Delete All Records

Example:- student_data = Student.objects.all().delete()

count() - It returns an integer representing the number of objects in the database matching the QuerySet. A count() call performs a SELECT COUNT(*) behind the scenes.

Example:-

student_data = Student.objects.all()

print(student_data.count())

Field Lookups

Field lookups are how you specify the meet of an SQL WHERE clause.

They're specified as keyword arguments to the QuerySet methods filter(), exclude() and get().

If you pass an invalid keyword argument, a lookup function will raise TypeError.

Syntax:- **field__lookuptype=value**

Example:- Student.objects.filter(marks__lt='50')

SELECT * FROM myapp_student WHERE marks < '50';

The field specified in a lookup has to be the name of a model field.

In case of a ForeignKey you can specify the field name suffixed with _id. In this case, the value parameter is expected to contain the raw value of the foreign model's primary key.

Example:-

Student.objects.filter(stu_id=10)

exact - Exact match. If the value provided for comparison is None, it will be interpreted as an SQL NULL. This is case sensitive

Example:- Student.objects.get(name__exact='sonam')

iexact - Exact match. If the value provided for comparison is None, it will be interpreted as an SQL NULL. This is case insensitive

Example:- Student.objects.get(name__iexact='sonam')

contains - Case-sensitive containment test.

Example:- Student.objects.get(name__contains='kumar')

icontains - Case-insensitive containment test.

Example:- Student.objects.get(name__icontains='kumar')

in - In a given iterable; often a list, tuple, or queryset. It's not a common use case, but strings (being iterables) are accepted.

Example:- Student.objects.filter(id__in=[1, 5, 7])

gt - Greater than.

Example: - Student.objects.filter(marks__gt=50)

gte - Greater than or equal to.

Example: - Student.objects.filter(marks__gte=50)

lt - Less than.

Example: - Student.objects.filter(marks__lt=50)

lte - Less than or equal to.

Example: - Student.objects.filter(marks__lte=50)

startswith - Case-sensitive starts-with.

Example: - Student.objects.filter(name__startswith='r')

istartswith - Case-insensitive starts-with.

Example: - Student.objects.filter(name__istartswith='r')

endswith - Case-sensitive ends-with.

Example:- Student.objects.filter(name__endswith='j')

iendswith - Case-insensitive ends-with.

Example:- Student.objects.filter(name__iendswith='j’)

range - Range test (inclusive).

Example:- Student.objects.filter(passdate__range=('2020-04-01', '2020-05-05'))

SQL: - SELECT ... WHERE admission_date BETWEEN '2020-04-01' and '2020-05-05';

You can use range anywhere you can use BETWEEN in SQL — for dates, numbers and even characters.

date - For datetime fields, casts the value as date. Allows chaining additional field lookups. Takes a date value.

Example:-

Student.objects.filter(admdatetime__date=date(2020, 6, 5))

Student.objects.filter(admdatetime__date__gt=date(2020, 6, 5))

year - For date and datetime fields, an exact year match. Allows chaining additional field lookups. Takes an integer year.

Example:-

Student.objects.filter(passdate__year=2020)

Student.objects.filter(passdate__year__gt=2019)

month - For date and datetime fields, an exact month match. Allows chaining additional field lookups. Takes an integer 1 (January) through 12 (December).

Example:-

Student.objects.filter(passdate__month=6)

Student.objects.filter(passdate__month__gt=5)

day - For date and datetime fields, an exact day match. Allows chaining additional field lookups. Takes an integer day.

Example:-

Student.objects.filter(passdate__day=5)

Student.objects.filter(passdate__day__gt=3)

This will match any record with a pub_date on the third day of the month, such as January 3, July 3, etc.

week - For date and datetime fields, return the week number (1-52 or 53) according to ISO-8601, i.e., weeks start on a Monday and the first week contains the year's first Thursday.

Example:-

Student.objects.filter(passdate__week=23)

Student.objects.filter(passdate__week__gt=22)

week_day - For date and datetime fields, a 'day of the week' match. Allows chaining additional field lookups.

Takes an integer value representing the day of week from 1 (Sunday) to 7 (Saturday).

Example:-

Student.objects.filter(passdate__week_day=6)

Student.objects.filter(passdate__week_day__gt=5)

This will match any record with a admission_date that falls on a Monday (day 2 of the week), regardless of the month or year in which it occurs. Week days are indexed with day 1 being Sunday and day 7 being Saturday.

time - For datetime fields, casts the value as time. Allows chaining additional field lookups. Takes a datetime.time value.

Example:- Student.objects.filter(admdatetime__time__gt=time(6,00))

hour - For datetime and time fields, an exact hour match. Allows chaining additional field lookups. Takes an integer between 0 and 23.

Example:- Student.objects.filter(admdatetime__hour__gt=5)

minute - For datetime and time fields, an exact minute match. Allows chaining additional field lookups. Takes an integer between 0 and 59.

Example:- Student.objects.filter(admdatetime__minute__gt=50)

second - For datetime and time fields, an exact second match. Allows chaining additional field lookups. Takes an integer between 0 and 59.

Example:- Student.objects.filter(admdatetime__second__gt=30)

isnull - Takes either True or False, which correspond to SQL queries of IS NULL and IS NOT NULL, respectively.

Example:- Student.objects.filter(roll__isnull=False)

Limiting QuerySets

Use a subset of Python's array-slicing syntax to limit your QuerySet to a certain number of results. This is the equivalent of SQL's LIMIT and OFFSET clauses.

Student.objects.all()[:5] - This returns First 5 objects

Student.objects.all()[5:10] - This returns sixth through tenth objects

Student.objects.all()[-1] - This is not valid.

Student.objects.all()[:10:2] - This returns a list of every second object of the first 10.

bulk_create(objs, batch_size=None, ignore_conflicts=False) – This method inserts the provided list of objects into the database in an efficient manner.

It casts objs to a list, which fully evaluates objs if it's a generator. The cast allows inspecting all objects so that any objects with a manually set primary key can be inserted first.

The batch_size parameter controls how many objects are created in a single query. The default is to create all objects in one batch, except for SQLite where the default is such that at most 999 variables per query are used.

On databases that support it (all but Oracle), setting the ignore_conflicts parameter to True tells the database to ignore failure to insert any rows that fail constraints such as duplicate unique values. Enabling this parameter disables setting the primary key on each model instance

Example:-

```
objs = [
 Student(name='Sonal', roll=120, city='Dhanbad', marks=40, pass_date='2020-5-4'),
 Student(name='Kunal', roll=121, city='Dumka', marks=50, pass_date='2020-5-7'),
 Student(name='Anisa', roll=122, city='Giridih', marks=70, pass_date='2020-5-9') ]
student_data = Student.objects.bulk_create(objs)
```

Admin Application

It is a built-in application provided by Django.

This application provides admin interface for CRUD operations without writing sql statements.

It reads metadata from your models to provide a quick, model-centric interface where trusted users can manage content on your site.

Admin Application can be accessed using http://127.0.0.1:8000/admin

Super User is required to login into Admin Application

Create Super User

We need super user to login into admin interface of the admin application.

createsuperuser command is used to create super user.

Syntax:- **python manage.py createsuperuser**

How to Register Model

We are registering our table which we has created using model class, to default admin interface.

To Register Follow:-

- Open *admin.py* file which is inside Application Folder
- Import your own Model Class created inside Application's models.py
- admin.site.register(ModelClassName)

Example:-

- Open *admin.py*
- from enroll.models import Student
- admin.site.register(Student)

__str__( ) Method

The __str__() method is called whenever you call str() on an object. To display an object in the Django admin site and as the value inserted into a template when it displays an object. Thus, you should always return a nice, human-readable representation of the model from the __str__() method.

Write this Method in your own model class which is inside models.py file.

Syntax:-

```
def __str__(self):
        return self.fieldName
```

Example:-

```
def __str__(self):
        return self.stuname
```

ModelAdmin

The ModelAdmin class is the representation of a model in the admin interface.

To show table's all data in admin interface we have to create an ModelAdmin class in admin.py file of Application folder.

Syntax:-

Creating Class

```
Class ModelAdminClassName(admin.ModelAdmin):
        list_display=('fieldname1', 'fieldname2', …….)
```

Register Above Created Class

```
admin.site.register(ModelClassName, ModelAdminClassName)
```

Example: -

```
class StudentAdmin(admin.ModelAdmin):
        list_display=('id', 'stuid', 'stuname')
admin.site.register(Student, StudentAdmin)
```

list_display

Set list_display to control which fields are displayed on the change list page of the admin. If you don't set list_display, the admin site will display a single column that displays the __str__() representation of each object

There are four types of values that can be used in list_display:

- The name of a model field.
- A callable that accepts one argument, the model instance.
- A string representing a ModelAdmin method that accepts one argument, the model instance.
- A string representing a model attribute or method (without any required arguments).

Register Model by Decorator

A decorator can be used to register ModelAdmin Classes.

Syntax:- @admin.register(ModelClassName1, ModelClassName2,… ,site=custom_admin_site)

Register Model Classes

@admin.register(ModelClassName)

Creating Class

Class ModelAdminClassName(admin.ModelAdmin):

    list_display=('fieldname1', 'fieldname2', …….)

Example: -

@admin.register(Student)

class StudentAdmin(admin.ModelAdmin):

    list_display=('id', 'stuid', 'stuname')

Aggregation

sometimes you will need to retrieve values that are derived by summarizing or aggregating a collection of objects.

aggregate() - It is a terminal clause for a QuerySet that, when invoked, returns a dictionary of name-value pairs. The name is an identifier for the aggregate value; the value is the computed aggregate. The name is automatically generated from the name of the field and the aggregate function.

Syntax:- aggregate(name=agg_function('field'), name=agg_function('field'),)

field - It describes the aggregate value that we want to compute.

name - If you want to manually specify a name for the aggregate value, you can do so by providing that name when you specify the aggregate clause.

Django provides the following aggregation functions in the **django.db.models** module.

Avg(expression, output_field=None, distinct=False, filter=None, **extra) - It returns the mean value of the given expression, which must be numeric unless you specify a different output_field.

Default alias: <field>__avg

Return type: float if input is int, otherwise same as input field, or output_field if supplied

Has one optional argument:

distinct - If distinct=True, Avg returns the mean value of unique values. This is the SQL equivalent of AVG(DISTINCT <field>). The default value is False.

Count(expression, distinct=False, filter=None, **extra) - It returns the number of objects that are related through the provided expression.

Default alias: <field>__count

Return type: int

Has one optional argument:

distinct - If distinct=True, the count will only include unique instances. This is the SQL equivalent of COUNT(DISTINCT <field>). The default value is False.

Max(expression, output_field=None, filter=None, **extra)- It returns the maximum value of the given expression.

Default alias: <field>__max

Return type: same as input field, or output_field if supplied

Min(expression, output_field=None, filter=None, **extra) - It returns the minimum value of the given expression.

Default alias: <field>__min

Return type: same as input field, or output_field if supplied

Sum(expression, output_field=None, distinct=False, filter=None, **extra) - It computes the sum of all values of the given expression.

Default alias: <field>__sum

Return type: same as input field, or output_field if supplied

Has one optional argument:

distinct - If distinct=True, Sum returns the sum of unique values. This is the SQL equivalent of SUM(DISTINCT <field>). The default value is False

Django Form

Django's form functionality can simplify and automate vast portions of work like data prepared for display in a form, rendered as HTML, edit using a convenient interface, returned to the server, validated and cleaned up etc and can also do it more securely than most programmers would be able to do in code they wrote themselves.

Django handles three distinct parts of the work involved in forms:

- preparing and restructuring data to make it ready for rendering
- creating HTML forms for the data
- receiving and processing submitted forms and data from the client

Bound and Unbound Forms

If it's bound to a set of data, it's capable of validating that data and rendering the form as HTML with the data displayed in the HTML.

If it's unbound, it cannot do validation (because there's no data to validate!), but it can still render the blank form as HTML.

Create Django Form using Form Class

To create Django form we have to create a new file inside application folder lets say file name is **forms.py**. Now we can write below code inside **forms.py** to create a form:-

Syntax:-

```
from django import forms

class FormClassName(forms.Form):

        label=forms.FieldType()

        label=forms.FieldType(label='display_label')
```

Example:-

```
from django import forms

class StudentRegistration(forms.Form):

        name=forms.CharField()

        email=forms.EmailField
```

form class that converted to html code

```
<tr>

  <th> <label for="id_name">Name:</label></th>

  <td> <input type="text" name="name" required
        id="id_name"> </td>

</tr>

<tr>

  <th> <label for="id_email">Email:</label></th>

  <td><input type="email" name="email" required
        id="id_email"></td>

</tr>
```

Display Form to User

- Create an object of Form class in **views.py** then pass object to template files
- Use Form object in template file

Creating Form object in views.py

First of all create form object inside **views.py** file then pass this object to template file as a dict.

views.py

```
from .forms import StudentRegistration
def showformdata(request):
        fm = StudentRegistration()
        return render(request, 'enroll/userregistration.html', {'form':fm})
```

Get object from views.py in template file

templates/enroll / userregistration.html

```
<!DOCTYPE html>
<html>
        <body>
                {{form}}
        </body>
</html>
```

Note - Form object won't provide form tag and button you have to write them manually in template file.

{{form}} doesn't add form tag and button so we have to add them manually.

templates/enroll / userregistration.html

```
<!DOCTYPE html>
<html>
        <body>
```

```
            <form action =“” method=“get”>
                {{form}}
                <input type=“submit” value=“Submit”>
            </form>
        </body>
</html>
```

Form Rendering Options

- {{form}} will render them all
- {{ form.as_table }} will render them as table cells wrapped in <tr> tags
- {{ form.as_p }} will render them wrapped in <p> tags
- {{ form.as_ul }} will render them wrapped in <li> tags

```
<form action =“” method=“get”>
        {{form}}
        <input type=“submit” value=“Submit”>
</form>

<form action =“” method=“get”>
        {{form.as_p}}
        <input type=“submit” value=“Submit”>
</form>
```

Ordering Form Field

order_fields(field_order) – This method is used to rearrange the fields any time with a list of field names as in field_order. By default Form.field_order=None, which retains the order in which you define the fields in your form class.

If field_order is a list of field names, the fields are ordered as specified by the list and remaining fields are appended according to the default order.

Unknown field names in the list are ignored.

fm = StudentRegistration()

fm.order_fields(field_order=['email', 'name'])

Render Form Field Manually

Each field is available as an attribute of the form using {{form.name_of_field}}

{{ field.label }} - The label of the field.

Example:- {{ form.name.label }}

{{ field.label_tag }} - The field's label wrapped in the appropriate HTML <label> tag. This includes the form's label_suffix. The default label_suffix is a colon:

Example:- {{ form.name.label_tag }}

{{ field.id_for_label }} - The ID that will be used for this field.

Example:- {{ form.name.id_for_label }}

{{ field.value }} - The value of the field.

Example:- {{ form.name.value }}

{{ field.html_name }} - The name of the field that will be used in the input element's name field. This takes the form prefix into account, if it has been set.

Example:- {{ form.name.html_name }}

{{ field.help_text }} - Any help text that has been associated with the field.

Example:- {{ form.name.help_text }}

{{ field.field }} - The Field instance from the form class that this BoundField wraps. You can use it to access Field attributes.

Example:- {{form.name.field.help_text}}

{{ field.is_hidden }} - This attribute is True if the form field is a hidden field and False otherwise. It's not particularly useful as a template variable, but could be useful in conditional tests such as:

```
{% if field.is_hidden %}
  {# Do something special #}
{% endif %}
```

{{ field.errors }} - It outputs a <ul class="errorlist"> containing any validation errors corresponding to this field. You can customize the presentation of the errors with a {% for error in field.errors %} loop. In this case, each object in the loop is a string containing the error message.

Example:- {{ form.name.errors }}

```
<ul class="errorlist">
   <li>Enter Your Name</li>
</ul>
```

{{ form.non_field_errors }} - This should be at the top of the form and the template lookup for errors on each field.

Example:- {{ form.non_field_errors }}

```
<ul class="errorlist nonfield">
   <li>Generic validation error</li>
</ul>
```

Form Field

A form's fields are themselves classes; they manage form data and perform validation when a form is submitted.

Syntax:- FieldType(**kwargs)

Example:-

IntegerField()

CharField(required)

CharField(required, widget=forms.PasswordInput)

```
from django import forms

class StudentRegistration(forms.Form):

 name = forms.CharField()
```

label - The label argument lets you specify the “human-friendly” label for the field. This is used when the Field is displayed in a Form.

help_text - The help_text argument lets you specify descriptive text for this Field. If you provide help_text, it will be displayed next to the Field when the Field is rendered.

error_messages - The error_messages argument lets you override the default messages that the field will raise. Pass in a dictionary with keys matching the error messages you want to override.

Example:- name=forms.CharField(error_messages={'required':'Enter Your Name’})

widget - The widget argument lets you specify a Widget class to use when rendering the Field.

Widget

A widget is Django’s representation of an HTML input element.

The widget handles the rendering of the HTML, and the extraction of data from a GET/POST dictionary that corresponds to the widget.

The HTML generated by the built-in widgets uses HTML5 syntax, targeting <!DOCTYPE html>.

Whenever you specify a field on a form, Django will use a default widget that is appropriate to the type of data that is to be displayed.

Each field type has an appropriate default Widget class, but these can be overridden as required.

Form fields deal with the logic of input validation and are used directly in templates.

Widgets deal with rendering of HTML form input elements on the web page and extraction of raw submitted data.

Example:-

TextInput

Textarea

attrs - A dictionary containing HTML attributes to be set on the rendered widget.

If you assign a value of True or False to an attribute, it will be rendered as an HTML5 boolean attribute.

Example:-

feedback=forms.CharField(widget=forms.TextInput(attrs={'class':'somecss1 somecss2', 'id':'uniqueid'}))

TextInput - It renders as: <input type="text" ...>

Example:- name = forms.CharField(widget=forms.TextInput)

NumberInput - It renders as: <input type="number" ...>

EmailInput - It renders as: <input type="email" ...>

URLInput – It renders as: <input type="url" ...>

PasswordInput - It renders as: <input type="password" ...>

It take one optional argument *render_value* which determines whether the widget will have a value filled in when the form is re-displayed after a validation error (default is False).

DateInput - It renders as: <input type="text" ...>

It take one optional argument *format*. The format in which this field's initial value will be displayed.

If no *format* argument is provided, the default format is the first format found in DATE_INPUT_FORMATS.

CheckboxInput - It renders as: <input type="checkbox" ...>

It takes one optional argument *check_test* which is a callable that takes the value of th

Select - It renders as: <select><option ...>...</select>

choices attribute is optional when the form field does not have a choices attribute. If it does, it will override anything you set here when the attribute is updated on the Field.

NullBooleanSelect - Select widget with options 'Unknown', 'Yes' and 'No'

SelectMultiple - Similar to Select, but allows multiple selection: <select multiple>...</select>

RadioSelect - Similar to Select, but rendered as a list of radio buttons within <li> tags:

```
<ul>
  <li><input type="radio" name="..."></li>
  ...
</ul>
```

GET and POST

- GET should be used only for requests that do not affect the state of the system.
- Any request that could be used to change the state of the system should use POST.
- GET would also be unsuitable for a password form, because the password would appear in the URL, and thus, also in browser history and server logs, all in plain text. Neither would it be suitable for large quantities of data, or for binary data, such as an image. A Web application that uses GET requests for admin forms is a security risk: it can be easy for an attacker to mimic a form's request to gain access to sensitive parts of the system.
- POST, coupled with other protections like Django's CSRF protection offers more control over access.
- GET, by contrast, bundles the submitted data into a string, and uses this to compose a URL. The URL contains the address where the data must be sent, as well as the data keys and values.
- Django's login form is returned using the POST method, in which the browser bundles up the form data, encodes it for transmission, sends it to the server, and then receives back its response.
- GET is suitable for things like a web search form, because the URLs that represent a GET request can easily be bookmarked, shared, or resubmitted.

What is Cross Site Request Forgery (CSRF/ XSRF)

A Cross-site request forgery hole is when a malicious site can cause a visitor's browser to make a request to your server that causes a change on the server. The server thinks that because the request comes with the user's cookies, the user wanted to submit that form.

Depending on which forms on your site are vulnerable, an attacker might be able to do the following to your victims:

- Log the victim out of your site.
- Change the victim's site preferences on your site.
- Post a comment on your site using the victim's login.
- Transfer funds to another user's account.

Attacks can also be based on the victim's IP address rather than cookies:

- Post an anonymous comment that is shown as coming from the victim's IP address.

- Modify settings on a device such as a wireless router or cable modem.
- Modify an intranet wiki page.
- Perform a distributed password-guessing attack without a botnet.

Django provides CSRF Protection with csrf_token which we need to add inside form tag. This token will add a hidden input field with random value in form tag.

templates/enroll / userregistration.html

```
<!DOCTYPE html>
<html>
    <body>
        <form method="post"> {% csrf_token %}
            {{form}}
            <input type="submit" value="Submit">
        </form>
    </body>
</html>
```

Get Django Form Data

- Validate Data/ Field Validation
- Get Cleaned Data

How to send GET Request

- Open browser and write url hit enter this is by default get request
- Anchor Tag
- Form tag contains method='GET'
- Form tag with specifying method is by default GET

How to send POST Request

- form tag contains method='POST'

Django Form and Field Validation

is_valid( ) - This method is used to run validation and return a Boolean designating whether the data was valid as True or not as False. This returns True or False.

Syntax:- Form.is_valid()

cleaned_data – This attribute is used to access clean data. Each field in a Form class is responsible not only for validating data, but also for "cleaning" it normalizing it to a consistent format. This is a nice feature, because it allows data for a particular field to be input in a variety of ways, always resulting in consistent output. Once you've created a Form instance with a set of data and validated it, you can access the clean data via its cleaned_data attribute.

Any text based field such as CharField or EmailField always cleans the input into a string.

If your data does not validate, the cleaned_data dictionary contains only the valid fields.

cleaned_data will always only contain a key for fields defined in the Form, even if you pass extra data when you define the Form.

When the Form is valid, cleaned_data will include a key and value for all its fields, even if the data didn't include a value for some optional fields.

Get Django Form Data in Terminal

1. First of all Create form inside forms.py file

forms.py

from django import forms

```
class StudentRegistration(forms.Form):
        name=forms.CharField()
        email=forms.EmailField()
```

2. Get submitted Data in views.py

views.py

```
from .forms import StudentRegistration
def showformdata(request):
        if request.method=='POST':
                fm = StudentRegistration(request.POST)
                if fm.is_valid():
                        print('Form Validated')
                        print('Name:', fm.cleaned_data['name'])
                        print('email:', fm.cleaned_data['email'])
        else:
                fm = StudentRegistration()
        return render(request, 'enroll/userregistration.html', {'form':fm})
```

3. Get object from views.py in template file

templates/enroll / userregistration.html

```
<!DOCTYPE html>
<html>
        <body>
                <form method="POST"> {% csrf_token%}
                        {{form.as_p}}
                        <input type="submit" value="Submit">
                </form>
        </body>
```

</html>

Form Field

A form's fields are themselves classes; they manage form data and perform validation when a form is submitted.

Syntax:- FieldType(**kwargs)

Example:-

IntegerField()

CharField(required)

CharField(required, widget=forms.PasswordInput)

from django import forms

class StudentRegistration(forms.Form):

name = forms.CharField()

CharField(**kwargs)

Default widget: TextInput

Empty value: Whatever you've given as empty_value.

Normalizes to: A string.

Uses MaxLengthValidator and MinLengthValidator if max_length and min_length are provided. Otherwise, all inputs are valid.

Error message keys: required, max_length, min_length

It has three optional arguments for validation:

max_length and *min_length* - If provided, these arguments ensure that the string is at most or at least the given length.

strip - If True (default), the value will be stripped of leading and trailing whitespace.

empty_value - The value to use to represent "empty". Defaults to an empty string.

Example:- name=forms.CharField(min_length=5, max_length=50, error_messages={'required':'Enter Your Name'})

IntegerField(**kwargs)

Default widget: NumberInput when Field.localize is False, else TextInput.

Empty value: None

Normalizes to: A Python integer.

Validates that the given value is an integer. Uses MaxValueValidator and MinValueValidator if max_value and min_value are provided. Leading and trailing whitespace is allowed, as in Python's int() function.

Error message keys: required, invalid, max_value, min_value

The max_value and min_value error messages may contain %(limit_value)s, which will be substituted by the appropriate limit.

It takes two optional arguments for validation:

max_value and *min_value* - These control the range of values permitted in the field.

EmailField(**kwargs)

Default widget: EmailInput

Empty value: '' (an empty string)

Normalizes to: A string.

Uses EmailValidator to validate that the given value is a valid email address, using a moderately complex regular expression.

Error message keys: required, invalid

It has two optional arguments for validation, *max_length* and *min_length*. If provided, these arguments ensure that the string is at most or at least the given length.

DateField(**kwargs)

Default widget: DateInput

Empty value: None

Normalizes to: A Python datetime.date object.

Validates that the given value is either a datetime.date, datetime.datetime or string formatted in a particular date format.

ImageField(**kwargs)

Default widget: ClearableFileInput

Empty value: None

Normalizes to: An UploadedFile object that wraps the file content and file name into a single object.

Validates that file data has been bound to the form. Also uses FileExtensionValidator to validate that the file extension is supported by Pillow.

Error message keys: required, invalid, missing, empty, invalid_image

Using an ImageField requires that Pillow is installed with support for the image formats you use.

Cleaning and Validating Specific Field

clean_<fieldname>() - This method is called on a form subclass where <fieldname> is replaced with the name of the form field attribute.

This method does any cleaning that is specific to that particular attribute, unrelated to the type of field that it is.

This method is not passed any parameters.

You will need to look up the value of the field in self.cleaned_data and remember that it will be a Python object at this point, not the original string submitted in the form.

forms.py

```
from django import forms

class StudentRegistration(forms.Form):
        name=forms.CharField()
        email=forms.EmailField()
        password=forms.CharField(widget=forms.PasswordInput)
        def clean_name(self):
                valname = self.cleaned_data['name']   //                valname =
self.cleaned_data.get('name')
                if len(valname) < 4:
```

```
                raise forms.ValidationError('Enter more than or equal 4')
        return valname
```

Validation of Complete Django Form at once

clean() – The clean() method on a Field subclass is responsible for running to_python(), validate(), and run_validators() in the correct order and propagating their errors.

If, at any time, any of the methods raise ValidationError, the validation stops and that error is raised.

This method returns the clean data, which is then inserted into the cleaned_data dictionary of the form.

Implement a clean() method on your Form when you must add custom validation for fields that are interdependent.

Syntax:- Form.clean()

forms.py

```
from Django import forms
class StudentRegistration(forms.Form):
    name=forms.CharField()
    email=forms.EmailField()
    def clean(self):
        cleaned_data = super().clean()
        valname=self.cleaned_data['name']
        if len(valname)<4:
            raise forms.ValidationError('Name should be more than or equal 4')
        valemail=self.cleaned_data['email']
        if len(valemail)<10:
            raise forms.ValidationError('Email should be more than or equal 10')
```

Using Built-in Validators

We can use Built-in Validators, available in django.core module.

forms.py

```
form django.core import validators
from django import forms
class StudentRegistration(forms.Form):
        name=forms.CharField(validators=[validators.MaxLengthValidator(10)])
        email=forms.EmailField()
```

Create Custom Form Validators

forms.py

```
form django.core import validators
from django import forms
def starts_with_s(value):
    if vaule[0] != 's':
      raise forms.ValidationError('Name should start with s')
class StudentRegistration(forms.Form):
      name=forms.CharField(validators=[starts_with_s])
      email=forms.EmailField()
```

Model Form

Django provides a helper class that lets you create a Form class from a Django model. This helper class is called as ModelForm.

ModelForm is a regular Form which can automatically generate certain fields.

The fields that are automatically generated depend on the content of the Meta class and on which fields have already been defined declaratively.

Steps:-

- Create Model Class
- Create ModelForm Class

Syntax:-

forms.py

```
class ModelFormClassName(forms.ModelForm):
    class Meta:
        model = ModelClassName
        fields = [‘fieldname1', ‘fieldname2', ‘fieldname3’]
```

example

```
class Registration(forms.ModelForm):
```

```
    class Meta:
        model = User
        fields = [‘name’, ‘password’, ‘email']
```

models.py

```
from django.db import models
class User(models.Model):
 name = models.CharField(max_length=70)
 email = models.EmailField(max_length=100)
 password = models.CharField(max_length=100)
```

forms.py

```
from django.core import validators
from django import forms
from .models import User
class StudentRegistration(forms.ModelForm):
 class Meta:
  model = User
  fields = ['name', 'password', 'email']
```

views.py

```
def showformdata(request):
 if request.method == 'POST':
  fm = StudentRegistration(request.POST)
  if fm.is_valid():
   nm = fm.cleaned_data['name']
   em = fm.cleaned_data['email']
   pw = fm.cleaned_data['password']
```

```
  reg = User(name=nm, email=em, password=pw)
  reg.save()
```

templatefile.html

```
<form action="" method="POST" novalidate>
 {% csrf_token %}
  {{form.as_p}}
  <input type="submit" value="Submit">
 </form>
```

Authentication and Authorization

Authentication – Authentication is about validating your credentials like Username and password to verify your identity.

Authorization – Authorization is the process to determine whether the authenticated user has access to the particular resources. It checks your rights to grant you access to resources such as information, databases, files, etc.

User Authentication System

Django comes with a user authentication system. It handles user accounts, groups, permissions and cookie-based user sessions.

Django authentication provides both authentication and authorization together and is generally referred to as the authentication system.

By default, the required configuration is already included in the settings.py generated by django-admin startproject, these consist of two items listed in your INSTALLED_APPS setting:

'django.contrib.auth' contains the core of the authentication framework, and its default models.

'django.contrib.contenttypes' is the Django content type system, which allows permissions to be associated with models you create.

and these items in your MIDDLEWARE setting:

SessionMiddleware manages sessions across requests.

AuthenticationMiddleware associates users with requests using sessions.

django.contrib.auth

C:\Users\RK\AppData\Local\Programs\Python\Python38-32\Lib\site-packages\django\contrib\auth

models.py

class User

User Object - User objects are the core of the authentication system.

They typically represent the people interacting with your site and are used to enable things like restricting access, registering user profiles, associating content with creators etc.

Only one class of user exists in Django's authentication framework, i.e., 'superusers' or admin 'staff' users are just user objects with special attributes set, not different classes of user objects.

User object Fields

username - Usernames may contain alphanumeric, _, @, +, . and - characters. Its required and length is 150 characters or fewer.

first_name – Its optional (blank=True) and length is 30 characters or fewer

last_name – Its optional (blank=True) and length is 150 characters or fewer.

email – Its optional (blank=True)

password – A hash of, and metadata about, the password. Django doesn't store the raw password. Its required.

groups - Many-to-many relationship to Group.

user_permissions - Many-to-many relationship to Permission.

is_staff - Designates whether this user can access the admin site. It takes Boolean.

is_active - Designates whether this user account should be considered active. We recommend that you set this flag to False instead of deleting accounts; that way, if your applications have any foreign keys to users, the foreign keys won't break. It takes Boolean.

is_superuser - Designates that this user has all permissions without explicitly assigning them. It takes Boolean.

last_login - A datetime of the user's last login.

date_joined - A datetime designating when the account was created. Is set to the current date/time by default when the account is created.

is_authenticated - This is a way to tell if the user has been authenticated. This does not imply any permissions and doesn't check if the user is active or has a valid session. Its a read-only attribute which is always True.

is_anonymous - This is a way of differentiating User and AnonymousUser objects. It's a read-only attribute which is always False

authenticate ( )

authenticate(request=None, **credentials) – This is used to verify a set of credentials. It takes credentials as keyword arguments, username and password for the default case, checks them against each authentication backend, and returns a User object if the credentials are valid for a backend.

If the credentials aren't valid for any backend or if a backend raises PermissionDenied, it returns None.

request is an optional HttpRequest which is passed on the authenticate() method of the authentication backends.

Example:-

user = authenticate(username='sonam', password='geekyshows')

login ( )

login(request, user, backend=None) - To log a user in, from a view, use login(). It takes an HttpRequest object and a User object. login() saves the user's ID in the session, using Django's session framework.

When a user logs in, the user's ID and the backend that was used for authentication are saved in the user's session. This allows the same authentication backend to fetch the user's details on a future request.

logout ( )

logout(request) - To log out a user who has been logged in via django.contrib.auth.login(), use django.contrib.auth.logout() within your view. It takes an HttpRequest object and has no return value.

When you call logout(), the session data for the current request is completely cleaned out. All existing data is removed. This is to prevent another person from using the same Web browser to log in and have access to the previous user's session data.

Type of Views

- Function Based View
- Class Based View

Class Based View

Class-based views provide an alternative way to implement views as Python objects instead of functions.

They do not replace function-based views.

- Base Class-Based Views / Base View
- Generic Class-Based Views / Generic View

Advantages:-

- Organization of code related to specific HTTP methods (GET, POST, etc.) can be addressed by separate methods instead of conditional branching.
- Object oriented techniques such as mixins (multiple inheritance) can be used to factor code into reusable components.

Class Based View:-

```
from django.views import View

class MyView(View):

    def get(self, request):

        return HttpResponse('<h1>Class Based View</h1>')


from django.urls import path

from school import views

urlpatterns = [

     path('cl/', views.MyView.as_view(), name='cl'),

]
```

Model Inheritance

Model inheritance in Django works almost identically to the way normal class inheritance works in Python. The base class should subclass django.db.models.Model.

- Abstract Base Classes
- Multi-table Inheritance
- Proxy Models

Abstract Base Classes

Abstract base classes are useful when you want to put some common information into a number of other models.

You write your base class and put *abstract=True* in the *Meta* class.

This model will then not be used to create any database table. Instead, when it is used as a base class for other models, its fields will be added to those of the child class.

It does not generate a database table or have a manager, and cannot be instantiated or saved directly.

Fields inherited from abstract base classes can be overridden with another field or value, or be removed with None.

```
from django.db import models

class CommonInfo(models.Model):
    name = models.CharField(max_length=70)
    age = models.IntegerField()
    class Meta:
        abstract = True
class Student(CommonInfo):
    fees = models.IntegerField()
class Teacher(CommonInfo):
    salary = models.IntegerField()
```

Meta Inheritance - When an abstract base class is created, Django makes any Meta inner class you declared in the base class available as an attribute.

If a child class does not declare its own Meta class, it will inherit the parent's Meta.

If the child wants to extend the parent's Meta class, it can subclass it.

Django does make one adjustment to the Meta class of an abstract base class: before installing the Meta attribute, it sets abstract=False.

This means that children of abstract base classes don't automatically become abstract classes themselves.

You can make an abstract base class that inherits from another abstract base class. You just need to remember to explicitly set abstract=True each time.

```
from django.db import models

class CommonInfo(models.Model):
    name = models.CharField(max_length=70)
```

```
    age = models.IntegerField()
    class Meta:
        abstract = True
class Student(CommonInfo):
    fees = models.IntegerField()
    class Meta:
        abstract = False
```

Multi-table Inheritance

In this inheritance each model have their own database table, which means base class and child class will have their own table.

The inheritance relationship introduces links between the child model and each of its parents (via an automatically-created OneToOneField).

```
from django.db import models
class ExamCenter(models.Model):
  cname = models.CharField(max_length=70)
  city = models.CharField(max_length=70)
class Student(ExamCenter):
 name = models.CharField(max_length=70)
 roll = models.IntegerField()
```

All of the fields of ExamCenter will also be available in Student, although the data will reside in a different database table.

Proxy Model

Sometimes, however, you only want to change the Python behavior of a model – perhaps to change the default manager, or add a new method.

This is what proxy model inheritance is for: creating a proxy for the original model. You can create, delete and update instances of the proxy model and all the data will be saved as if you were using the original (non-proxied) model. The difference is that you can change things like the default model ordering or the default manager in the proxy, without having to alter the original.

Proxy models are declared like normal models. You tell Django that it's a proxy model by setting the proxy attribute of the Meta class to True.

```
from django.db import models

class ExamCenter(models.Model):
  cname = models.CharField(max_length=70)
  city = models.CharField(max_length=70)



class MyExamCenter(ExamCenter):
 class Meta:
   proxy = True
   ordering = ['city']
 def do_something(self):
    pass
```

- A proxy model must inherit from exactly one non-abstract model class.
- You can't inherit from multiple non-abstract models as the proxy model doesn't provide any connection between the rows in the different database tables.
- A proxy model can inherit from any number of abstract model classes, providing they do not define any model fields.

- A proxy model may also inherit from any number of proxy models that share a common non-abstract parent class.
- If you don’t specify any model managers on a proxy model, it inherits the managers from its model parents.
- If you define a manager on the proxy model, it will become the default, although any managers defined on the parent classes will still be available.